

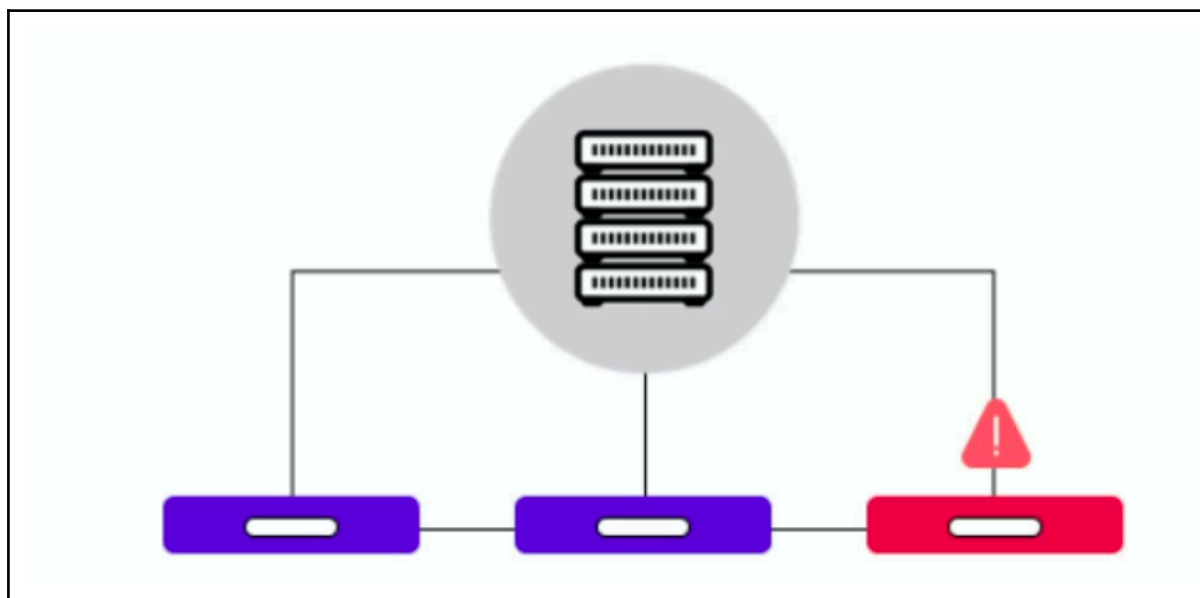
Section 7: System Performance

Introduction to System Performance

In this section, we will explore how modern systems stay operational despite failures by mastering the core principles of reliability, high availability, fault tolerance, and disaster recovery. And let's get started with introducing system reliability.

What is System Reliability?

Reliability is the ability of a system to operate continuously without failure. Reliability is one of those qualities that users notice when everything is working fine but they immediately notice when it doesn't. At its core, reliability is a system's ability to continue operating correctly over time, even as conditions change and failures occur. That goes beyond simply staying online.



A reliable system produces correct results, behaves consistently across requests, tolerates component failures without causing widespread outages, and maintains high uptime so that the users can depend on it whenever they need it. Think of these as complementary qualities rather than separate goals. A system that is always available but returns incorrect data is not reliable. Likewise, a system that works perfectly but crashes frequently is not reliable either. True reliability is achieved only when all of these characteristics work together and deliver a dependable user experience.

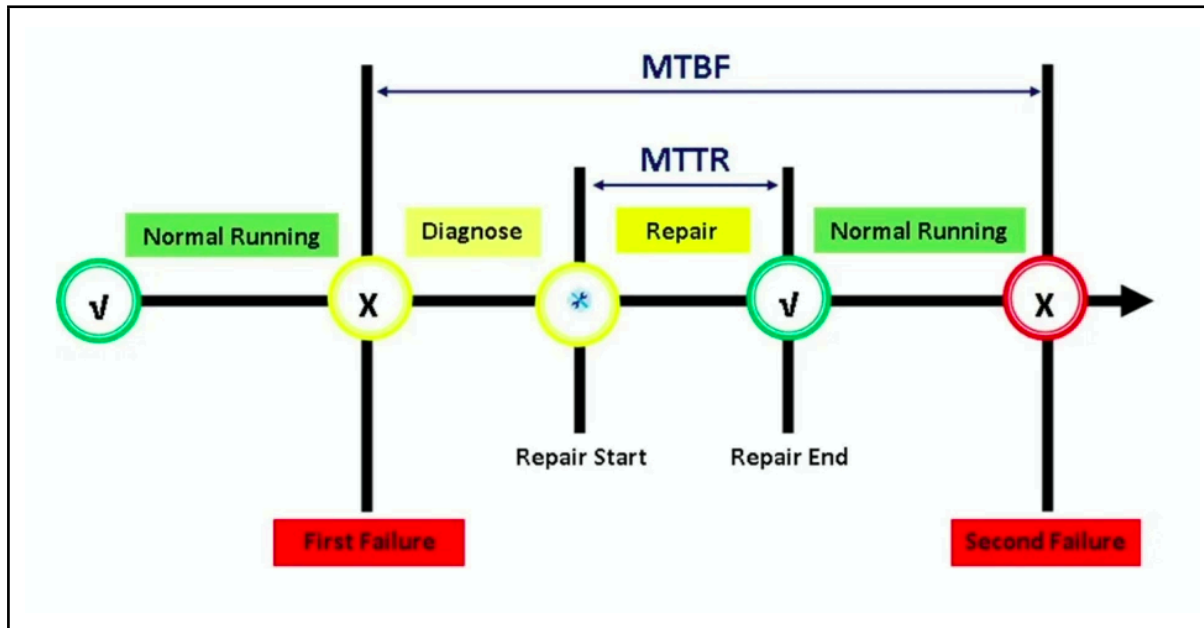
Why Reliability Matters?

As systems grow, reliability stops being nice-to-have and becomes a business requirement. Users don't think about architecture. They simply expect your application to be available, responsive, and dependable every time they use it. Whether it is online banking, e-commerce, or messaging, even a few minutes of downtime can lead to financial loss, damaged reputation, and most importantly, lost user trust. But reliability is not about building systems that never fail. That is unrealistic.



In distributed systems, failures, servers crash, networks become unreliable and dependencies go down. What separates a well-designed system is how gracefully it handles those failures and how quickly it recovers from them. That is the mindset that we build throughout this section. Don't design for a world where nothing fails, but design for a world where failures are expected, contained, and recovered, and recovered from with minimal impact to your users.

Key Metrics - MTBF & MTTR



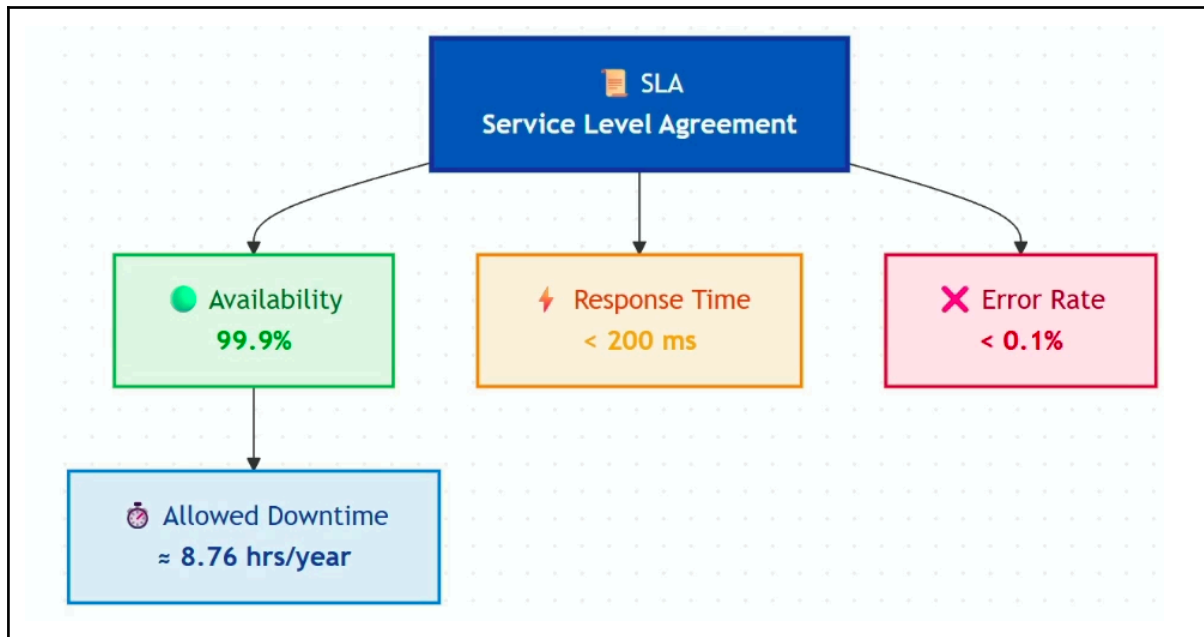
When engineers talk about reliability, they don't rely on opinions. They rely on measurable metrics. Two of the most important are MTBF and MTTR.

- **MTBF:** MTBF or Mean Time Between Failures, measures how long a system operates before experiencing a failure. A higher MTBF generally indicates a more stable and more resilient system.
- **MTTR:** MTTR or Mean Time to Recovery, measures how quickly the system can be restored after a failure occurs. Since failures are inevitable in distributed systems, reducing recovery time is just as important as preventing failures in the first place.

The most reliable systems strike balance between these two metrics. They fail infrequently, and when they do, they recover rapidly with minimal impact on the users. That is why improving reliability is not just about reducing failures. It is equally about designing for fast and efficient recovery.

What are SLAs (Service Level Agreements)?

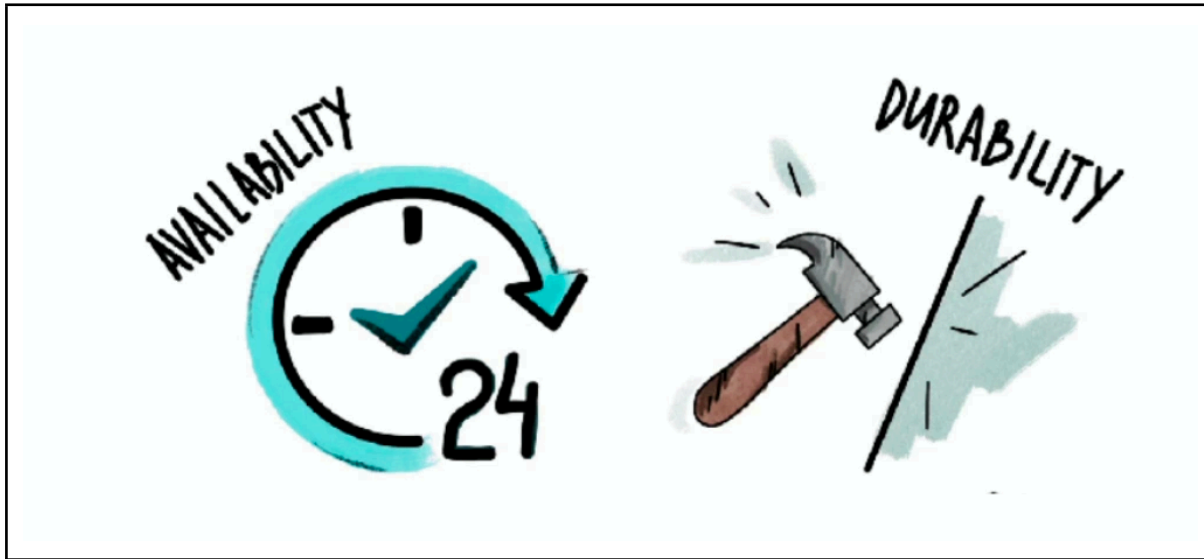
An SLA defines the level of service that a provider commits to delivering. It is more than just a technical target. It is a measurable promise to customers, backed by clear performance expectations. The most common SLA metrics include availability, response time and error rate. These metrics set expectations for how often the system should be accessible, how quickly it should respond, and how reliably it should process the request.



Now it is very important to understand these numbers, what they really mean. A 99.9% availability target sounds extremely high, but it allows for around 8.76 hours of downtime over the course of 1 year. That is why many mission-critical systems aim for even higher availability targets such as 99.99 or even 9.9999 even.

As architects, SLAs help us translate business expectations into engineering goals. They directly influence the design decisions that we make around redundancy, monitoring, failover, and recovery to ensure that the system consistently meets its promised level of service.

Availability vs. Durability



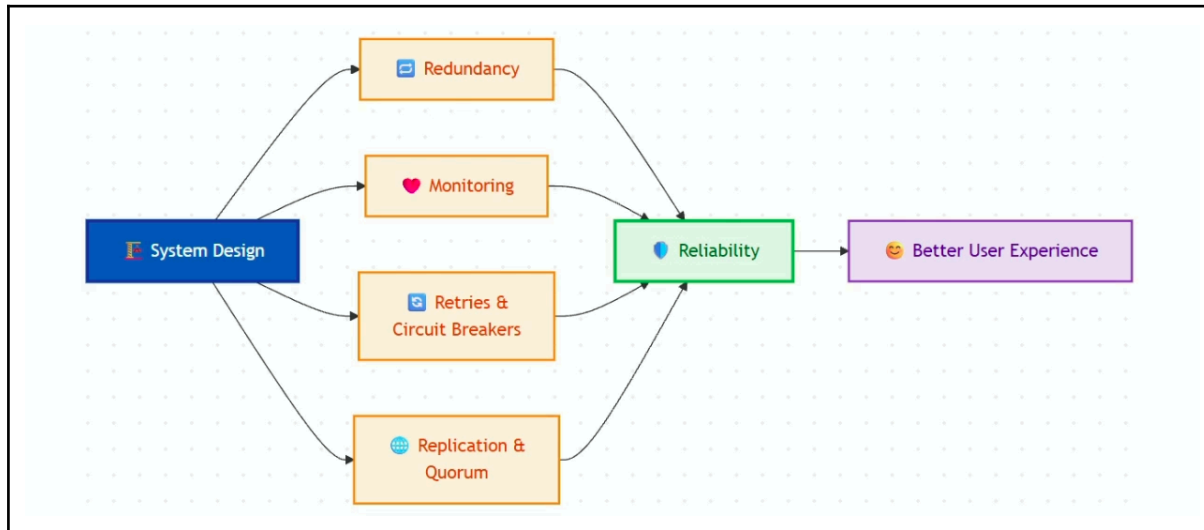
Availability and durability are often mentioned together but they solve two very different problems. As architects, it is not to confuse them. Availability answers the question, can users access the system right now? And if the application is online, responsive, and serving requests, it is considered available.

Durability on the other hand, answers a different question. Is the data still safe? Even if a server crashes or the entire service goes offline, durable systems ensure that committed data is not lost or corrupted.

A useful way to think is like this. Availability protects the user experience, while availability protects the user's data. Ideally, you want both. A highly available system that loses data is unacceptable, and a perfectly durable system that is constantly offline is not very useful either. Building reliable systems means designing for both continuous service and long-term data protection.

Impact of Reliability on System Design

Reliability is not something that you add at the end of the project. It is something that you design for from the very beginning. Every major architectural decision influences how well a system behaves when failures inevitably happen.

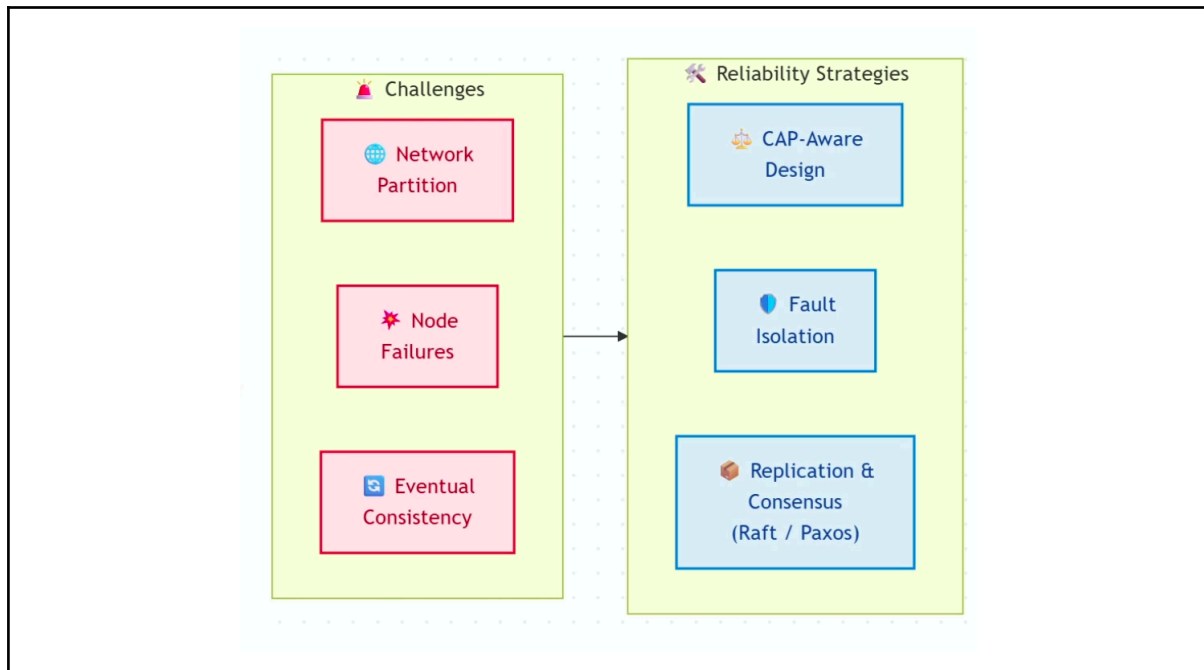


For example, redundancy eliminates single points of failure by running multiple instances and enabling automatic failover. Then health check and monitoring help to detect problems early so that recovery can begin before users are significantly affected. And when systems communicate with external services, retry mechanisms can recover from temporary issues, while circuit breakers prevent repeated failures from spreading across the entire system.

And in distributed systems, techniques like replication and quorum ensure that data remains available and resilient even if individual nodes become unavailable. As you will see throughout this section, reliable systems are not built by relying on a single technique. They are the result of multiple architectural patterns working together to anticipate failures, isolate their impact, and recover quickly with minimal disruption to the users.

Reliability in Distributed Systems

The distributed system introduces a new reality here, failures are no longer exceptional but are expected. Unlike a single server application, you now have multiple machines communicating over networks that can be slow, unreliable, or even temporarily unavailable. Common challenges include network partitions that isolate parts of the system, node failures that take services offline, and the need to balance consistency and availability here.

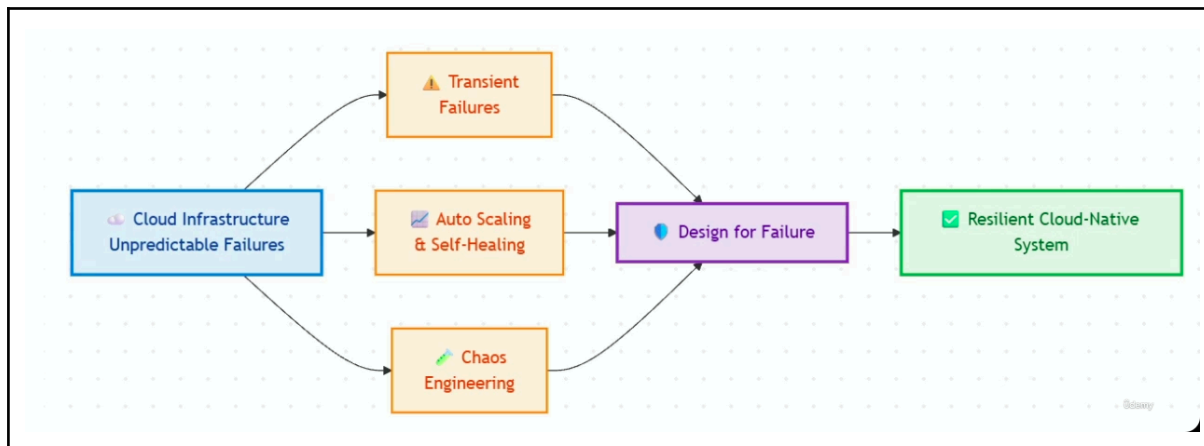


Now in many large-scale systems, you will also encounter eventual consistency, where updates propagate across replicas over time rather than appearing everywhere instantly. Designing reliable distributed systems means embracing these realities of trying to eliminate them. That starts with making gap-aware design decisions, isolating failures so that they don't cascade across the system, and replicating critical data to avoid single points of failure. And for operations that need multiple nodes to agree on a shared state, consensus algorithms such as Paxos and Raft provide a safe and coordinated way to do so, even when failures occur.

The key takeaway is very simple. In distributed systems, reliability is not achieved by preventing failures, it is actually achieved by designing systems that continue operating correctly despite failures.

Reliability in Cloud-native Systems

Now cloud-native architectures require a very different way of thinking about reliability. In traditional environments, failures were treated as rare events. In the cloud, there are more expected events. In the cloud, virtual machines can disappear, containers can restart, networks can experience temporary issues, and services can become unavailable without warning.



That is why cloud-native applications are built to handle transient failures automatically. Instead of failing immediately, they use techniques like retries, timeouts, and circuit breakers to recover from short-lived issues. They also rely on auto-scaling to handle changing workloads and self-healing mechanisms that automatically replace unhealthy instances without any manual intervention. But there is one more important practice - testing your assumptions. Chaos engineering deliberately introduces controlled failures into production-like environments to verify that the system behaves as expected. It helps teams to discover weaknesses before real users experience these issues. This leads to one of the most important principles in modern architecture - don't design systems assuming everything will work. Design them assuming that something will eventually fail. When you embrace this kind of mindset, your system becomes far more resilient in real-world production environments.

Summary & Key Takeaways

- Reliability is foundational to user trust
- Metrics like MTBF, MTTR, SLAs are vital to quantify it
- Design choices must anticipate and tolerate failure
- In distributed/cloud systems, reliability is an active design challenge
- What's next:
 - High Availability, Fault Tolerance & Failover

Interview Questions - System Reliability

- Conceptual Questions:
 - What is system reliability, and why is it important in system design?
 - Explain the difference between availability and durability with real-world examples.
 - What are MTBF and MTTR? How do they relate to each other?
 - How do SLAs help define system reliability expectations?
- Practical/Scenario-Based:
 - How would you design a system to ensure 99.99% availability?
 - Imagine one of your microservices goes down frequently. How would you identify and fix reliability issues?
 - How would you improve reliability in a system with high user traffic and data volume?
 - Describe how redundancy and failover can be applied in cloud-native systems to improve reliability.
- Behavioral/Trade-off Questions:
 - Tell me about a time you had to choose between performance and reliability.
 - How would you ensure high reliability without over-engineering a system?